

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE

Department of Computer Science Engineering

Course Code: 23CS002PC304

Course Title: AI Assisted Coding

Year/Sem: III / II

Assignment Number: 18.3 / 24

Name: O.Narasimha Raju

HT No: 2303A53034

Batch: 46

Lab Objectives

- Solve advanced graph problems using AI assistance
- Implement maximum flow and matching algorithms
- Analyze graph-based optimization problems
- Apply algorithms to real-world scenarios

Task 1: Maximum Flow (Ford-Fulkerson)

Python Code

```
from collections import deque

def bfs(capacity, source, sink, parent):
    visited = [False]*len(capacity)
    queue = deque([source])
    visited[source] = True

    while queue:
        u = queue.popleft()
```

```

        for v in range(len(capacity)):
            if not visited[v] and capacity[u][v] > 0:
                queue.append(v)
                visited[v] = True
                parent[v] = u
                if v == sink:
                    return True
        return False

def max_flow(capacity, source, sink):
    parent = [-1]*len(capacity)
    flow = 0

    while bfs(capacity, source, sink, parent):
        path_flow = float('inf')
        s = sink

        while s != source:
            path_flow = min(path_flow, capacity[parent[s]][s])
            s = parent[s]

        v = sink
        while v != source:
            u = parent[v]
            capacity[u][v] -= path_flow
            capacity[v][u] += path_flow
            v = parent[v]

        flow += path_flow

    return flow

capacity = [
    [0,0,10,0,0],
    [0,0,0,5,3],
    [0,0,0,7,6],
    [0,0,0,0,10],
    [0,0,0,0,0]
]

```

```
print("Maximum Flow =", max_flow(capacity, 0, 4))
```

Task 2: Maximum Matching (Bipartite)

Python Code

```
def bpm(u, graph, match, visited):
    for v in range(len(graph[u])):
        if graph[u][v] and not visited[v]:
            visited[v] = True
            if match[v] == -1 or bpm(match[v], graph, match, visited):
                match[v] = u
                return True
    return False

def max_matching(graph):
    match = [-1]*len(graph[0])
    result = 0

    for u in range(len(graph)):
        visited = [False]*len(graph[0])
        if bpm(u, graph, match, visited):
            result += 1

    return result

graph = [
    [1,1,0],
    [1,0,1],
    [0,1,0]
]

print("Maximum Matching =", max_matching(graph))
```

Task 3: Applications

- Network traffic optimization
- Job assignment problems
- Resource allocation systems

Conclusion

Advanced graph algorithms solve complex optimization problems.

AI tools help implement these algorithms efficiently and accurately.